Tutorial: Apache NetBeans for Java CLA

Introduction

As we move into the later parts of the course, we will be exploring **GUI (Graphical User Interface) programming** with Java, likely using frameworks like Swing or JavaFX. Apache NetBeans has exceptionally strong, integrated visual tools (like a "drag-and-drop" designer) that make building GUIs much simpler and more intuitive than in VS Code.

This tutorial will help you get comfortable with NetBeans by using it for something you already know **creating a command-line application**. We'll cover creating a project, exploring the IDE, writing code, debugging, and finally packaging your application into a distributable JAR file.

What you will learn:

- How to create a new Java with Maven project in NetBeans.
- The basic layout of the NetBeans IDE.
- How to write, run, and debug your command-line application.
- How to package your project into a runnable JAR file.

Part 1: Prerequisites

Before we start, please ensure you have the following installed:

- **JDK (Java Development Kit):** You should already have this from your work in VS Code. A JDK version 17, 21 or later is recommended.
- **Apache NetBeans:** Download the latest version from the official website: https://netbeans.apache.org/download/. The installer is straightforward; simply follow the on-screen instructions.

Part 2: Creating Your First Command-Line Project

We'll start by creating a standard "Hello, World!" application. NetBeans is project-oriented, which helps keep all your files, libraries, and configurations organized. We will use **Maven**, a powerful build and dependency management tool that is fully integrated into NetBeans.

1. **Launch Apache NetBeans.**
2. **Create a New Project:**

- Go to the menu bar and click File -> New Project....
- A "New Project" wizard will appear.
- In the **Categories** pane, select **Java with Maven**.
- In the **Projects** pane, select **Java Application**.
- Click **Next**.

3. **Configure the Project:**

- **Project Name:** Give your project a name. Let's use MyFirstNetBeansApp.
- **Project Location:** Choose a folder where you want to save your project.
- **Group Id:** This is a unique identifier for your project's group, conventionally written in reverse domain name format. For now, you can use something like com.mycourse.
- **Version:** You can leave this as the default 1.0-SNAPSHOT.
- **Package:** NetBeans automatically suggests a package name based on your Group Id and Project Name (e.g., com.mycourse.myfirstnetbeansapp). This is where your Java source code will reside. It's good practice to keep this up.
- Click **Finish**.

NetBeans will now create the project structure. This might take a few seconds as it downloads the necessary Maven components for the first time.

Part 3: Exploring the NetBeans IDE

Once your project is created, the main window will look something like this. Let's break down the key areas, comparing them to what you might know from VS Code.

Projects Window (Left): This is similar to the Explorer view in VS Code. It shows a logical tree of all your projects and their files.

- **MyFirstNetBeansApp**: This is your project root.
- **Source Packages**: This is where your .java source files are. You'll see the package you created (com.mycourse.myfirstnetbeansapp) and the main Java file inside it.
- **Dependencies**: This folder shows external libraries your project uses (e.g., JUnit for testing). In Maven projects, this is managed automatically.
- **Project Files**: This contains the build configuration file, pom.xml. You can think of this as being similar to package.json in the web world or a .vscode configuration folder, but much more powerful.

Editor Window (Center): This is where you write your code. It has all the features you'd expect: syntax highlighting, code completion (press Ctrl+Space), error checking, and more. NetBeans automatically generated a main class for you.

Navigator Window (Bottom-Left): This provides a quick overview of the members (fields and methods) of the currently active Java class. You can double-click a method name here to jump directly to it in the editor.

Output Window (Bottom): This is your terminal/console. When you run your application, the output (System.out.println) will appear here. Build messages, errors, and debugger information also show up in different tabs within this window.

Part 4: Writing, Running, and Debugging Code

Let's modify the auto-generated code to do something more interesting.

Step 1: Write Some Code

Double-click MyFirstNetBeansApp.java in the Projects window to open it in the editor. Replace the existing main method with the following code. This simple program will greet the user and count from 1 to 5.

```
package com.mycourse.myfirstnetbeansapp;
import java.util.Scanner;
public class MyFirstNetBeansApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Please enter your name: ");
        String name = scanner.nextLine();
        System.out.println("Hello, " + name + "! Welcome to
NetBeans.");
        System.out.println("Let's count to 5:");
        for (int i = 1; i <= 5; i++) {
            System.out.println("Count: " + i);
        }
        System.out.println("Program finished.");
    }
}
```

Notice how NetBeans automatically adds import java.util.Scanner; for you when you use the Scanner class.

If it doesn't, you can click the lightbulb icon next to the error or press Alt+Enter to get suggestions.

Step 2: Run the Application

Running your program is simple.

- **Method 1:** Click the green "Run Project" button (▶) in the toolbar.
- **Method 2:** Press the F6 key.
- **Method 3:** Right-click the project name in the Projects window and select Run.

The **Output** window will activate. You'll be prompted to enter your name. Click inside the Output window, type your name, and press Enter. The rest of the program's output will then be displayed.

Step 3: Debug the Application

Debugging in NetBeans is just as powerful as in VS Code. The concepts are the same: set breakpoints and step through your code.

- **Set a Breakpoint:** Click in the left margin of the editor, next to a line number. A red square will appear, indicating a breakpoint. Let's set one on the line that says for (int i = 1; i <= 5; i++).
- **Start the Debugger:**
  Click the "Debug Project" button (a green arrow with a bug icon) in the toolbar.
  Press Ctrl+F5.
  Right-click the project and select Debug.
- **Interact with the Debugger:** The program will run and stop at your breakpoint. The line will be highlighted in green. At the bottom of the IDE, new windows will appear: **Variables** and **Call Stack**. The **Variables** window shows the current values of all variables in scope (e.g., name, scanner). This is incredibly useful!
- Use the **Debugger Toolbar** (usually at the top) to control the execution:
- **Step Over (F8):** Execute the current line and move to the next one.
- **Step Into (F7):** If the current line is a method call, jump into that method's code.
- **Continue (F5):** Resume normal execution until the next breakpoint or the end of the program.
- Press **Step Over (F8)** a few times. Watch the Variables window as the variable i is created and its value changes with each loop iteration. This is the core of debugging.

Part 5: Packaging Your Application as a Runnable JAR

You've written and tested your application. Now, how do you share it with someone who doesn't have NetBeans? You package it into a **runnable JAR (Java Archive) file**. Maven makes this incredibly easy.

- **Clean and Build the Project:** In the Projects window, right-click on your project name (MyFirstNetBeansApp).
- Select **Clean and Build**.

What just happened?

- **Clean:** Deletes any previously compiled files and packages.
- **Build:** Compiles your .java files into .class files and then packages them into a JAR file.

You will see a BUILD SUCCESS message in the Output window.

Locate the JAR File: In the Projects window, expand the target folder. This folder is created by Maven to store all build outputs. Inside target, you will find your JAR file. It will be named something like MyFirstNetBeansApp-1.0-SNAPSHOT.jar.

Run the JAR from the Command Line:

Open a terminal or command prompt on your computer (e.g., PowerShell on Windows, Terminal on macOS/Linux).Navigate to your project's target directory. For example:

```
cd path/to/your/NetBeansProjects/MyFirstNetBeansApp/target
```

Run the JAR file using the java -jar command:

```
java -jar MyFirstNetBeansApp-1.0-SNAPSHOT.jar
```

Your command-line application will run directly in the terminal, just as it did in NetBeans' Output window! You can now share this single .jar file with anyone who has Java installed.

Conclusion

Congratulations! You have successfully created, run, debugged, and packaged a Java command-line application using Apache NetBeans. You've learned that while the interface is different from VS Code, the core concepts are the same. You now have a solid foundation for using NetBeans for the rest of this course. When we move on to GUI development, you will already be familiar with the project structure, editor, and build process, allowing you to focus on learning the new GUI-specific tools that make NetBeans shine.